

DATABASE SYSTEMS

DB Lecture 1

Introduction to Databases and Database Management Systems

Topics covered in this lecture:

- What is data, information, and a database?
- Limitations of traditional file-based systems
- Database Management Systems (DBMS) and their purpose
- Advantages and disadvantages of the DBMS approach
- Database system architecture and the three-schema model
- Data models: relational, hierarchical, network, object-oriented
- The Entity-Relationship (ER) model
- Database users and administrators

Database Systems Lecture Series
Foundations of Data Storage and Management

1. What is a Database?

A **database** is an organized, structured collection of related data that is stored electronically and can be easily accessed, managed, and updated. Data on its own is simply a set of raw facts — numbers, text, dates, or symbols — that have no meaning until they are organized into a context. When data is processed and given context, it becomes **information**, which is useful for decision-making.

Databases are everywhere in modern computing: a university stores student records in a database, a bank stores account and transaction data, an e-commerce site stores product catalogs and orders, and a hospital stores patient records. In every one of these cases, the underlying requirement is the same — store large amounts of related data reliably, make it easy to search and update, and ensure multiple people can use it safely at the same time.

1.1 Key Characteristics of a Database

- **Persistent:** data continues to exist even after the program that created it terminates.
- **Shared:** multiple users and applications can access the same data concurrently.
- **Structured:** data is organized according to a defined schema or model.
- **Self-describing:** a database contains, in addition to the data itself, a description of its own structure, called **metadata**, typically stored in a system catalog.
- **Managed:** access, integrity, and security are controlled by a dedicated piece of software.

1.2 Data vs. Information vs. Knowledge

It is useful to distinguish three related terms. **Data** are raw, unprocessed facts (e.g. the number 42). **Information** is data that has been processed to have meaning (e.g. "42 students enrolled"). **Knowledge** is the understanding gained by analyzing and interpreting information over time, often used to make predictions or decisions. A database primarily stores data, while a well-designed information system turns that data into useful information for its users.

2. Limitations of File-Based Systems

Before DBMSs became widespread, applications stored data directly in flat files managed by the operating system. Each application had its own set of files and its own file-handling code embedded in the program. While simple for small tasks, this approach breaks down quickly as data and the number of applications grow.

2.1 Common Problems

- **Data redundancy and inconsistency:** the same data (e.g. a customer's address) may be duplicated in several files. Updating one copy but not another leads to inconsistent data.
- **Difficulty accessing data:** retrieving data that was not anticipated when the program was written often requires writing an entirely new program.
- **Data isolation:** data is scattered across different files in different formats, making it hard to write applications that need data from several sources.
- **Integrity problems:** business rules (e.g. "account balance cannot be negative") are buried in application code rather than enforced centrally, so they are easy to violate or duplicate incorrectly across programs.
- **Atomicity problems:** if a program fails midway through an operation (e.g. a funds transfer), the file system may be left in an inconsistent state.
- **Concurrent access anomalies:** when multiple users update the same file at the same time without coordination, updates can be lost or data can become corrupted.
- **Security problems:** it is difficult to restrict access to only part of the data (e.g. letting a clerk see order data but not salary data) when everything lives in plain files.

These problems motivated the development of the **Database Management System**, a layer of software specifically designed to solve them in a general, reusable way.

3. What is a DBMS?

A **Database Management System (DBMS)** is software that enables users to define, create, maintain, and control access to a database. It acts as an intermediary between the user (or application program) and the physical database, hiding the low-level details of how data is actually stored on disk.

3.1 Core Functions of a DBMS

- 1 **Data definition:** allows the creation, modification, and removal of definitions for the structure of data (tables, columns, constraints).
- 2 **Data manipulation:** allows insertion, updating, deletion, and retrieval of data through a query language such as SQL.
- 3 **Data security and authorization:** ensures only authorized users can access or modify specific data.
- 4 **Data integrity:** enforces rules that keep the data accurate and consistent (e.g. uniqueness, referential integrity).
- 5 **Concurrency control:** manages simultaneous access so that transactions do not interfere with one another.
- 6 **Backup and recovery:** restores the database to a correct state after a failure such as a crash or disk error.
- 7 **Data dictionary / catalog management:** maintains metadata that describes the structure of the data.

3.2 Examples of DBMS Products

Category	Examples
Relational (RDBMS)	MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server
Document (NoSQL)	MongoDB, CouchDB
Key-Value (NoSQL)	Redis, DynamoDB
Wide-Column (NoSQL)	Apache Cassandra, HBase
Graph	Neo4j, Amazon Neptune

4. Advantages and Disadvantages of the DBMS Approach

4.1 Advantages

- **Controlled redundancy:** data is centralized, reducing duplication and the risk of inconsistency.
- **Data consistency:** when redundancy is controlled, updates are reflected everywhere automatically.
- **Improved data sharing:** the same database can serve many applications and users at once.
- **Improved data security:** access control can be defined at a fine-grained level (table, column, or even row level).
- **Enforced integrity constraints:** rules can be defined once, centrally, and applied consistently.
- **Improved data accessibility:** declarative query languages like SQL let users retrieve data without writing procedural code.
- **Reduced application development time:** many common data-management tasks are already handled by the DBMS.
- **Backup and recovery services:** built-in mechanisms protect against data loss.

4.2 Disadvantages

- **Cost:** commercial DBMS software, hardware, and skilled personnel can be expensive.
- **Complexity:** DBMSs are large pieces of software requiring specialized knowledge to administer effectively.
- **Performance overhead:** the generality of a DBMS can add overhead compared to a specialized file-based solution built for one narrow purpose.
- **Single point of failure:** centralizing data means that if the DBMS becomes unavailable, all dependent applications are affected (mitigated with replication and clustering).

5. Database System Architecture: The Three-Schema Model

To achieve **data independence** — the ability to change the structure of the database at one level without affecting the levels above it — database systems are commonly organized using the ANSI/SPARC three-schema architecture.

5.1 The Three Levels

- **Internal (physical) level:** describes how data is actually stored — file organization, indexes, storage structures. This is closest to the hardware.
- **Conceptual (logical) level:** describes what data is stored and the relationships among it, independent of physical storage details. This is the level database designers work at.
- **External (view) level:** describes the part of the database that a particular group of users or an application is interested in. Multiple external views can exist over a single conceptual schema.

5.2 Data Independence

Physical data independence is the ability to change the internal schema (e.g. switch storage engines or add an index) without needing to change the conceptual schema or any application programs.

Logical data independence is the ability to change the conceptual schema (e.g. add a new table or attribute) without requiring changes to existing external views or the application programs that use them. Logical data independence is generally harder to achieve than physical data independence, since applications tend to be closely tied to the logical structure of the data they use.

6. Data Models

A **data model** is a collection of concepts that can be used to describe the structure of a database — the data types, relationships, and constraints that apply to the data. Most data models also include a set of basic operations for retrieving and updating data.

6.1 Categories of Data Models

- **Conceptual (high-level) models:** such as the Entity-Relationship (ER) model, provide concepts close to how users perceive data (entities, attributes, relationships).
- **Representational (implementation) models:** such as the relational model, provide concepts that are understandable to end users but not too far removed from how data is organized in storage.
- **Physical (low-level) models:** describe how data is stored as records, files, indexes, and pointers on disk.

6.2 Historical Data Models

Model	Key Idea
Hierarchical	Data organized as a tree; each child record has exactly one parent.
Network	Generalizes the hierarchical model; a record can have multiple parents (graph structure).
Relational	Data organized into tables (relations) of rows and columns; the dominant model since the 1980s.
Object-Oriented	Data represented as objects, similar to object-oriented programming languages.
NoSQL (various)	Document, key-value, column-family, and graph models designed for scale and flexible schemas.

7. The Entity-Relationship (ER) Model

The Entity-Relationship model is a conceptual data model widely used for high-level database design. It describes data in terms of **entities**, their **attributes**, and the **relationships** among them, and is typically visualized using an ER diagram.

7.1 Core Concepts

- **Entity:** a real-world object or concept that is distinguishable from other objects, e.g. a specific STUDENT or COURSE.
- **Entity type / entity set:** a collection of entities that share the same attributes, e.g. the set of all STUDENT entities.
- **Attribute:** a property that describes an entity, e.g. Name, Date of Birth, Student ID.
- **Key attribute:** an attribute (or set of attributes) whose value uniquely identifies each entity in an entity type, e.g. Student ID.
- **Composite attribute:** an attribute that can be broken down into smaller sub-parts, e.g. Address (Street, City, ZIP).
- **Multivalued attribute:** an attribute that can hold more than one value for an entity, e.g. a person's set of phone numbers.
- **Derived attribute:** an attribute whose value can be calculated from other attributes, e.g. Age derived from Date of Birth.
- **Relationship type:** an association among two or more entity types, e.g. ENROLLS_IN between STUDENT and COURSE.
- **Degree of a relationship:** the number of entity types participating; binary (2), ternary (3), and higher-degree relationships are all possible.
- **Cardinality ratio:** constrains the number of relationship instances an entity can participate in, expressed as one-to-one (1:1), one-to-many (1:N), or many-to-many (M:N).
- **Participation constraint:** specifies whether every entity must participate in the relationship (total participation) or only some do (partial participation).

7.2 A Simple Example

Consider a university database. STUDENT and COURSE are entity types. A STUDENT has attributes such as StudentID (key), Name, and DateOfBirth. COURSE has attributes such as CourseCode (key), Title, and Credits. The relationship type ENROLLS_IN connects STUDENT and COURSE with an M:N cardinality ratio, since a student can enroll in many courses and a course can have many students. The relationship itself may carry its own attribute, such as Grade or Semester, which does not belong naturally to either STUDENT or COURSE alone.

8. Database Users and Administrators

A database system is used and managed by several different categories of people, each interacting with the system in a different way and requiring different skills.

8.1 Categories of Users

- **Database Administrators (DBAs):** responsible for authorizing access, coordinating and monitoring use, acquiring resources, and controlling overall system performance.
- **Database Designers:** responsible for identifying the data to be stored and choosing appropriate structures to represent it, usually before the database is populated with data.
- **End users (naive/parametric):** use the database routinely through predefined applications, e.g. bank tellers or online shoppers, without knowledge of the underlying structure.
- **Sophisticated end users:** familiar with the database structure and query facilities, e.g. business analysts who write complex ad hoc SQL queries.
- **Application programmers:** write application programs that interact with the database, typically using an API or embedded query language.
- **System analysts and system designers:** determine the requirements of end users and develop specifications for canned transactions that meet those requirements.

8.2 Responsibilities of the DBA

- 1 Schema definition and modification.
- 2 Storage structure and access method definition.
- 3 Granting and revoking authorization to access the database.
- 4 Routine maintenance, including periodic backups, ensuring adequate free disk space, and monitoring jobs running against the database.
- 5 Setting and enforcing organizational policies for data usage and integrity.

8b. Database Schema vs. Database Instance

Two terms that are frequently confused by newcomers are **schema** and **instance**. Understanding the difference is essential for reasoning about how databases change over time.

8b.1 Schema

The **database schema** is the overall design of the database — the structure, including table names, column names, data types, and constraints. The schema is specified during database design and changes only occasionally, typically when the requirements of the application evolve. The schema is sometimes called the **intension** of the database.

8b.2 Instance

The **database instance** (also called the database state or snapshot) is the actual data stored in the database at a particular moment in time. Every time a row is inserted, updated, or deleted, the database moves from one valid instance to another, while the schema itself remains unchanged. The instance is sometimes called the **extension** of the database.

An analogy: the schema is like the blueprint of a house (walls, rooms, doors), while the instance is like a snapshot photograph of the house furnished at a particular moment. The blueprint changes rarely (renovation); the furniture arrangement (instance) can change daily.

8c. Database Languages

A DBMS provides several sub-languages, distinguished by the purpose they serve. These will be explored in depth in DB Lecture 2, but it is useful to see the big picture now.

8c.1 Data Definition Language (DDL)

The DDL is used to define and modify the database schema — creating, altering, and dropping tables and other schema objects. Statements such as **CREATE TABLE**, **ALTER TABLE**, and **DROP TABLE** belong to the DDL.

8c.2 Data Manipulation Language (DML)

The DML is used to retrieve, insert, update, and delete data within the tables defined by the schema. Statements such as **SELECT**, **INSERT**, **UPDATE**, and **DELETE** belong to the DML. DML can be further divided into **procedural DML**, where the user specifies what data is needed and how to get it, and **declarative (non-procedural) DML**, such as SQL, where the user specifies only what data is needed.

8c.3 Data Control Language (DCL)

The DCL governs permissions and access rights. Statements such as **GRANT** and **REVOKE** allow a DBA to control which users can perform which operations on which objects.

8c.4 Transaction Control Language (TCL)

The TCL manages the boundaries of transactions. Statements such as **COMMIT**, **ROLLBACK**, and **SAVEPOINT** allow a sequence of operations to be treated as a single unit of work, a topic explored in detail in DB Lecture 3.

```
-- A brief preview of SQL, covered fully in DB Lecture 2
CREATE TABLE Student (
  StudentID INT PRIMARY KEY,
  Name VARCHAR(50),
  DateOfBirth DATE
);

INSERT INTO Student VALUES (1, 'Amina Khan', '2002-05-14');

SELECT Name FROM Student WHERE StudentID = 1;
```

8d. DBMS System Architectures

Beyond the logical three-schema architecture, real database deployments also have a **system architecture** describing how the DBMS software, client applications, and storage are distributed across machines.

8d.1 Centralized Architecture

In a centralized architecture, the DBMS, the application, and the user interface all run on a single machine. This was typical of early mainframe database systems and is still used for small, single-user applications.

8d.2 Client-Server Architecture

In the client-server model, the DBMS runs on a dedicated **database server**, while **client** machines run the application logic and user interface, communicating with the server over a network using a query language such as SQL. This separation allows many clients to share one well-managed, powerful database server, and is the dominant architecture for business applications today.

- **Two-tier architecture:** the client communicates directly with the database server.
- **Three-tier architecture:** a middle application/business-logic tier sits between the client and the database server, common in web applications (browser → web/application server → database server).

8d.3 Distributed and Parallel Architectures

Large-scale systems may spread data and processing across multiple machines. A **distributed database** stores data across several physically separate sites that appear to users as a single logical database. A **parallel database** uses multiple processors and disks working together to improve performance for a single logical database, often within one data center. These architectures introduce additional challenges around consistency and coordination, which are explored briefly in DB Lecture 4.

8e. Review Questions

Use the following questions to check your understanding of the concepts introduced in this lecture before moving on to DB Lecture 2.

- 1 Explain, in your own words, the difference between data and information.
- 2 List four problems associated with traditional file-based data management and explain how a DBMS addresses each one.
- 3 What is the purpose of the three-schema architecture? Distinguish between physical and logical data independence.
- 4 Compare the hierarchical, network, and relational data models. Why did the relational model become dominant?
- 5 Define entity, attribute, and relationship in the context of the ER model, and give an original example of each.
- 6 What is the difference between a database schema and a database instance? Give an analogy of your own.
- 7 Describe the difference between DDL, DML, DCL, and TCL, giving one example statement for each.
- 8 Compare centralized, two-tier, and three-tier system architectures. Which is most common for modern web applications, and why?

9. Summary

In this lecture we introduced the fundamental motivation for database systems. We saw that traditional file-based systems suffer from redundancy, inconsistency, isolation, and security problems, and that a DBMS is designed specifically to overcome these issues by centralizing data definition, storage, security, and access control. We examined the three-schema architecture and the concept of data independence, surveyed the major families of data models, and introduced the Entity-Relationship model as a tool for conceptual database design. Finally, we looked at the different categories of people who interact with a database system.

Key Takeaways

- A DBMS solves the redundancy, consistency, security, and integrity problems inherent in file-based data storage.
- The three-schema architecture (internal, conceptual, external) provides physical and logical data independence.
- The relational model is the dominant data model in use today, though NoSQL models are important for specific use cases.
- The ER model is the standard tool for conceptual database design before implementation.

Looking Ahead

The next lecture, **DB Lecture 2**, dives into the relational model in detail: how data is represented as relations (tables), what keys and constraints are, and how the SQL language is used to define and manipulate relational data.

10. Glossary of Key Terms

Term	Definition
Database	An organized, persistent collection of related data.
DBMS	Software that defines, creates, and manages access to a database.
Metadata	Data that describes the structure of other data (the system catalog).
Schema	The overall logical structure/design of a database.
Instance	The actual data in a database at a given point in time.
Entity	A distinguishable real-world object represented in a database.
Attribute	A property that describes an entity.
Relationship	An association between two or more entities.
Data independence	The ability to change schema at one level without affecting schemas at other levels.
DBA	Database Administrator; manages and secures the database system.

This glossary is a quick reference; each term is discussed in more depth in the corresponding section of this lecture. Refer back to it as you progress through the rest of the DB Lecture series.